

Comunicação

STD29006 – Engenharia de Telecomunicações

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](https://creativecommons.org/licenses/by/4.0/)

Processos

Processo para o sistema operacional

- Para **cada processo** existe um **espaço de memória reservado**
- **Um processo não pode interferir** no funcionamento de outro processo
- **Escalonamento de processos** é uma atividade crucial de sistemas operacionais modernos
 - Estados: Pronto (*ready*), Em execução (*running*) e Em espera (*waiting*)
 - A **concorrência** entre processos é tratada de forma **transparente para o usuário**
 - chaveamento de contexto: CPU, alocação de segmento de memória, zerar segmento, etc.

Comunicação entre Processos (IPC)

- Arquivos

- Signals

- Pipe

- Sockets

```
echo -e "Fulano\nAna\nPedro" > alunos.txt  
  
cat alunos.txt
```

Comunicação entre Processos (IPC)

- Arquivos

- **Signals**

- Pipe

- Sockets

```
kill -9 PID
```

- [Veja aqui um exemplo em C](#) de como tratar sinais Unix

Comunicação entre Processos (IPC)

- Arquivos
- Signals
- Pipe
- Sockets

```
cat /etc/passwd | grep aluno
```

Comunicação entre Processos (IPC)

■ Arquivos

■ Signals

■ Pipe

■ Sockets

■ Para listar os sockets unix dos processos em execução

```
ss -lx  
  
# ou com o netstat (obsoleto)  
  
netstat -tln --unix
```


Threads

Processo pode ser **composto por diversas Threads**

- Cada thread possui um **pedaço independente de código** e não interfere no funcionamento de outras threads
- **Dados de um processo** podem ser **compartilhados** facilmente por **todas as threads**
- **Concorrência não é transparente** para o desenvolvedor
 - Contexto da thread: ready, running, waiting, blocked

Cliente multithread – Ex: navegador web

- Documento HTML é composto por texto e uma coleção de mídias
- Navegador web abre uma conexão TCP para obter cada elemento
- Cada elemento é exibido assim que é descarregado

Cliente multithread – Ex: navegador web

Inspector

Console

Debugger

{ }

Style Editor

@

Performance

Memory

Network

Storage

...

X

Filter URLs

All

HTML

CSS

JS

XHR

Fonts

Images

Media

WS

Other

☐ Persist Logs

☐ Disable cache

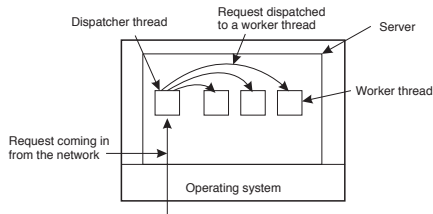
No throttling

HAR

Status	Method	File	Domain	Type	Transferred	Size	0 ms	320 ms	640 ms
200	GET	index.html	docente.ifsc.edu.br	html	3.52 KB	19.44 KB	→ 13 ms		
200	GET	android.css	docente.ifsc.edu.br	css	2.88 KB	9.54 KB	→ 4 ms		
200	GET	material.min.css	docente.ifsc.edu.br	css	20.39 KB	136.72 KB	→ 48 ms		
200	GET	docente.css	docente.ifsc.edu.br	css	988 B	1.99 KB	→ 13 ms		
200	GET	material.min.js	docente.ifsc.edu.br	js	11.66 KB	60.89 KB	→ 37 ms		
200	GET	icon?family=Material+Icons	fonts.googleapis.com	css	795 B	570 B	→ 66 ms		
301	GET	88x31.png	i.creativecommons.org	png	1.79 KB	1.43 KB	→ 17 ms		
304	GET	88x31.png	licensebuttons.net	png	cached	1.43 KB	→ 136 ms		
304	GET	analytics.js	www.google-analytics...js	cached	0 B		→ 85 ms		
200	GET	collect?v=1&_v=j68&a=199259187...	www.google-analytics...gif		444 B	35 B	→ 133 ms		

Servidor multithread – Ex: servidor de arquivos

- Operações de I/O são chamadas de sistema bloqueantes
- **Comportamento padrão do servidor de arquivos**
 - 1 Aguardar por pedido relacionado com uma operação com arquivos
 - 2 Processa o pedido e envia a resposta



- Servidor possui uma thread para esperar pedidos de clientes
 - Dispara uma thread para atender cada cliente

Servidor multithread – Exemplo em Java

```
public class Servidor{  
    public static void main(String args[]){  
        ServerSocket servidor = new ServerSocket(1234);  
        while(true){  
            Socket conexao = servidor.accept();  
            Thread t = new ServidorThread(conexao);  
            t.start();  
        }  
    }  
}
```

```
public class ServidorThread extends Thread{  
    private Socket conexao;  
    public ServidorThread(Socket c){  
        this.conexao = c;  
    }  
    public void run(){  
        System.out.println("Thread executada");  
    }  
}
```

Distribuição de tarefas

Aplicação multithread *versus* distribuída

■ Aplicação multithread

■ Processo principal (*main*)

- Responsável por criar e disparar as *threads* que vão executar as tarefas

■ Threads

- Responsáveis por executar as tarefas
- Podem compartilhar dados entre si

■ Aplicação distribuída

■ Processo coordenador (*main*)

- Responsável por distribuir tarefas, coordenar e compilar as respostas dos trabalhadores

■ Processos trabalhadores (*workers*)

- Responsáveis por processar tarefas e enviar os resultados para o coordenador
- Não compartilham dados entre si, apenas com o coordenador

Comunicação em Sistemas Distribu- dos

Comunicação entre cliente e servidor

- **Servidor**

- Processo que aguarda por pedidos de clientes

- **Cliente**

- Processo que inicia a comunicação com o servidor

- **Comunicação**

- Troca de mensagens entre cliente e servidor

- **Protocolo**

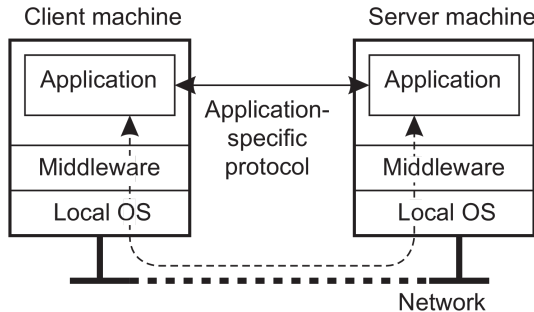
- Conjunto de regras que definem como a comunicação deve ocorrer

Comunicação entre processos é a essência de SD, pois processos são executados em diferentes máquinas

Comunicação entre cliente e servidor

Protocolo específico da camada de aplicação

- Cliente e servidor trocam mensagens de acordo com um protocolo
- Protocolo de comunicação é específico para cada aplicação
 - HTTP, FTP, SMTP, POP3, etc



Comunicação entre cliente e servidor

Processo servidor

- O servidor é um processo que aguarda por pedidos de clientes
 - Ao ser instanciado, deve-se especificar o endereço IP e a porta que ele irá escutar, bem como o protocolo de transporte
 - Ex: `127.0.0.1:8000 - tcp`
- Um processo pode ser associado a qualquer porta livre, porém é interessante evitar as portas padronizadas
 - 80 HTTP/TCP, 21 FTP/TCP, 25 SMTP/TCP
 - O comando `ss -ltn` pode ser usado para listar as portas TCP que possuem processos ouvindo
 - O arquivo `/etc/services` contém as portas padronizadas

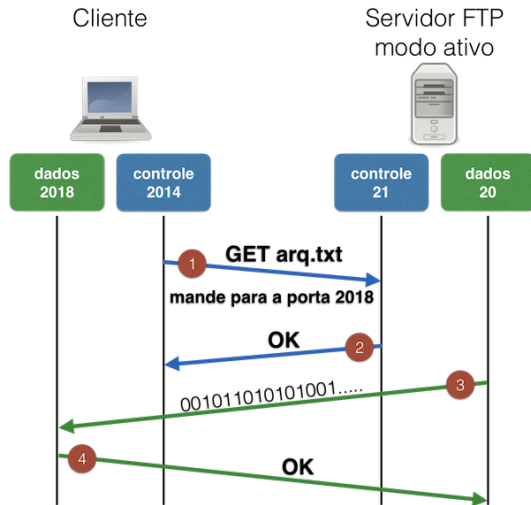
Protocolo de Transferência de Arquivos (FTP)

Comunicação entre cliente e servidor

- Porta de controle (21/TCP) a qual o cliente se conecta para enviar comandos
- **Modo ativo**
 - O servidor inicia a conexão com o cliente para transferir os dados
 - O cliente informa ao servidor qual porta ele irá escutar para transferir os dados
- **Modo passivo**
 - O cliente inicia ambas as conexões
 - O servidor informa ao cliente qual porta ele irá escutar para transferir os dados
 - Adequado para NATs e *firewalls*

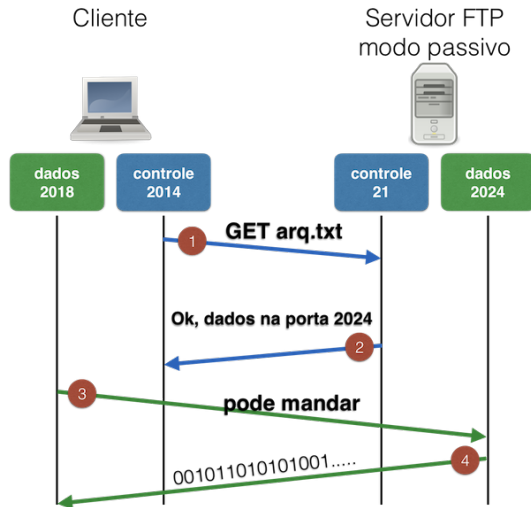
Protocolo de Transferência de Arquivos (FTP)

Modo ativo



Protocolo de Transferência de Arquivos (FTP)

Modo passivo



Manutenção do estado pelo servidor

■ Servidor que não mantém estado (*stateless server*)

- Não mantém qualquer informação sobre o cliente após atendê-lo e fechar a conexão
- Clientes e servidores são independentes, minimizando problemas de inconsistência de estado diante de uma falha do servidor
- Pode ter o desempenho prejudicado, uma vez que o servidor não consegue correlacionar pedidos subsequentes
 - Todo pedido é tratado como um pedido novo
 - Ex: Protocolo HTTP

Servidores e manutenção do estado

■ Servidor que mantém estado (*stateful server*)

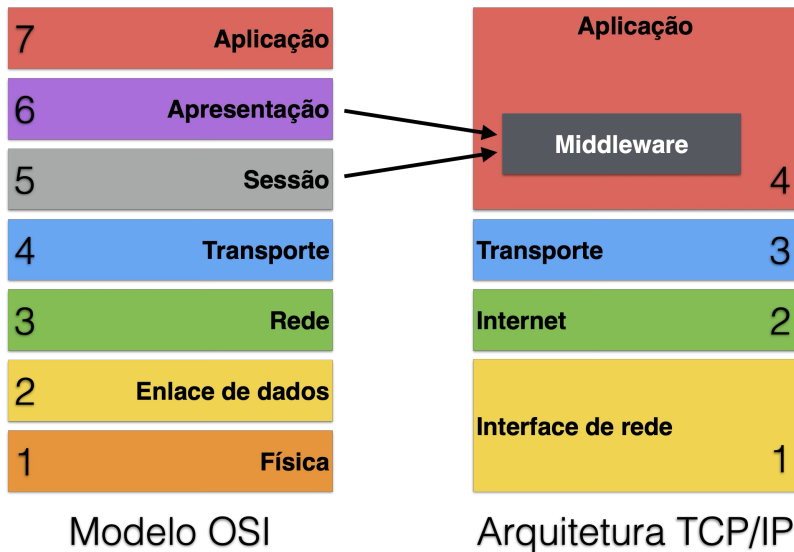
- Mantém informações sobre o cliente durante toda a comunicação
- Oferece um ótimo desempenho, uma vez que o servidor pode manter cópias locais das informações
- Dificuldades com confiabilidade (inconsistência), uma vez que o servidor pode falhar e perder as informações
 - Ex: Protocolo FTP

Middleware

Aplicação que provê um **conjunto de protocolos** de comunicação atuando como mediador entre processos clientes e servidores

- **Protocolos de alto nível** independentes de aplicações
- Provê suporte para transações, sincronização, protocolos de autenticação, etc

Middleware



Middleware – Tipos de comunicação

- **Persistência**

- **Sincronismo**

- **Fluxo**

Middleware – Tipos de comunicação

■ Persistência

- **Persistente** – mensagem fica armazenada no middleware o tempo que for necessário até ser entregue para o receptor – ex: e-mail
- **Transitória** – mensagem fica armazenada no middleware apenas enquanto emissor e receptor estiverem ativos – ex: tcp/udp

■ Sincronismo

■ Fluxo

Middleware – Tipos de comunicação

■ Persistência

- **Persistente** – mensagem fica armazenada no middleware o tempo que for necessário até ser entregue para o receptor – ex: e-mail
- **Transitória** – mensagem fica armazenada no middleware apenas enquanto emissor e receptor estiverem ativos – ex: tcp/udp

■ Sincronismo

- **Síncrono** – emissor fica bloqueado esperando resposta
- **Assíncrono** – não fica bloqueado e recebe uma notificação quando a resposta estiver disponível

■ Fluxo

Middleware – Tipos de comunicação

■ Persistência

- **Persistente** – mensagem fica armazenada no middleware o tempo que for necessário até ser entregue para o receptor – ex: e-mail
- **Transitória** – mensagem fica armazenada no middleware apenas enquanto emissor e receptor estiverem ativos – ex: tcp/udp

■ Sincronismo

- **Síncrono** – emissor fica bloqueado esperando resposta
- **Assíncrono** – não fica bloqueado e recebe uma notificação quando a resposta estiver disponível

■ Fluxo

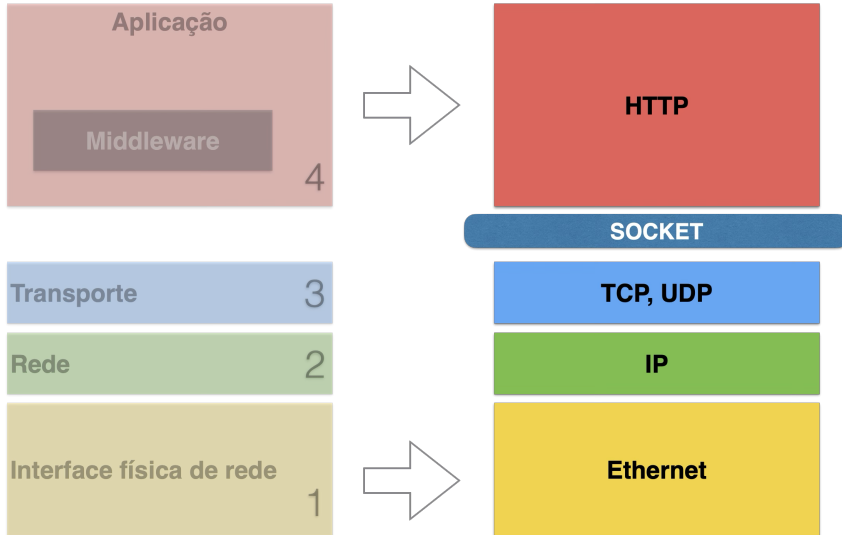
- **Discreto** – partes trocam mensagens, sendo cada mensagem tratada como uma unidade completa de informação – ex: navegação web
- **Fluxo contínuo** – são trocadas diversas mensagens consecutivas e estão relacionadas entre si – ex: rádio pela Internet

Modelo cliente/servidor

- Geralmente baseada no modelo de **comunicação síncrono** e com **persistência transitória**
- Cliente e servidor precisam estar ativos ao mesmo tempo e cliente fica bloqueado até receber a resposta

Sockets

Sockets



Cliente e Servidor com Sockets

Formas de comunicação entre processos no mesmo S.O.

- Arquivos, Signals, Pipe, etc.

```
ls -lR /etc/ | grep passwd
```

Cliente e Servidor com Sockets

Formas de comunicação entre processos no mesmo S.O.

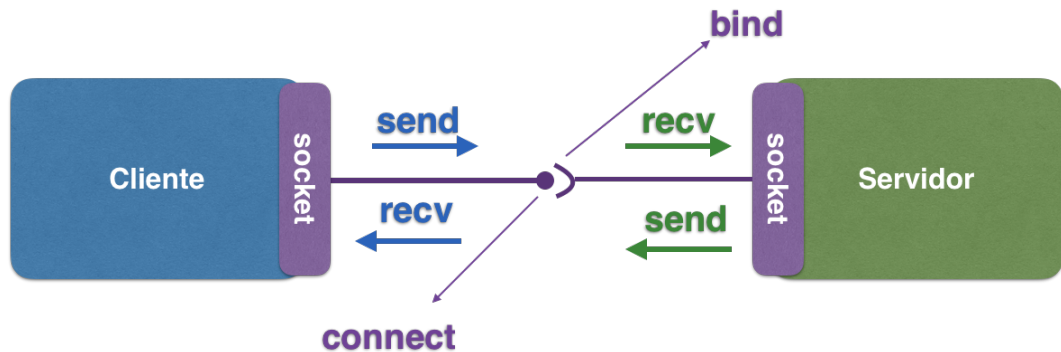
- Arquivos, Signals, Pipe, etc.

```
ls -lR /etc/ | grep passwd
```

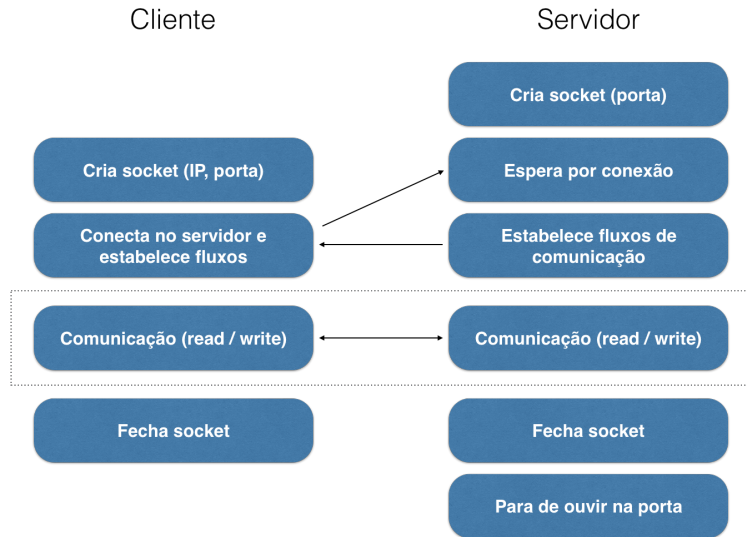
Sockets permite a **comunicação entre processos**, executados em **diferentes máquinas**

- API em C criada em 1983 no 4.2 BSD UNIX, é padrão em todos S.O.
- Sockets IP são identificados: protocolo de transporte, endereço IP e porta
 - TCP – Orientado a conexão
 - UDP – Orientado a datagramas (sem conexão)

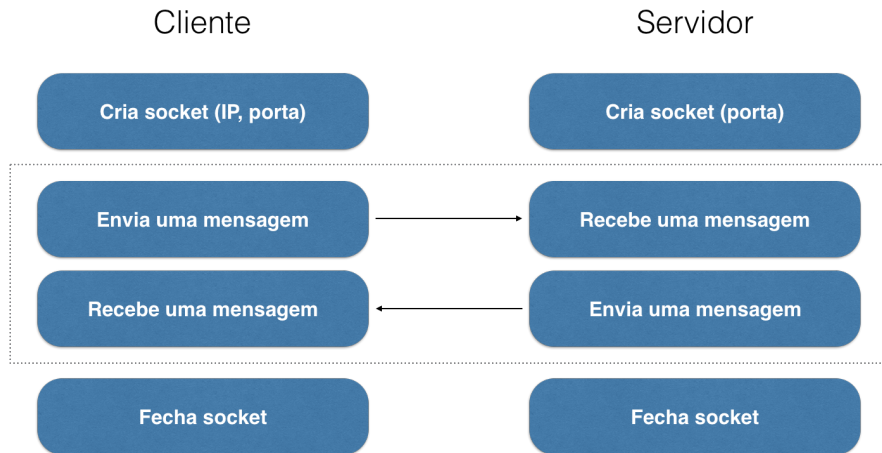
Cliente e Servidor com Sockets



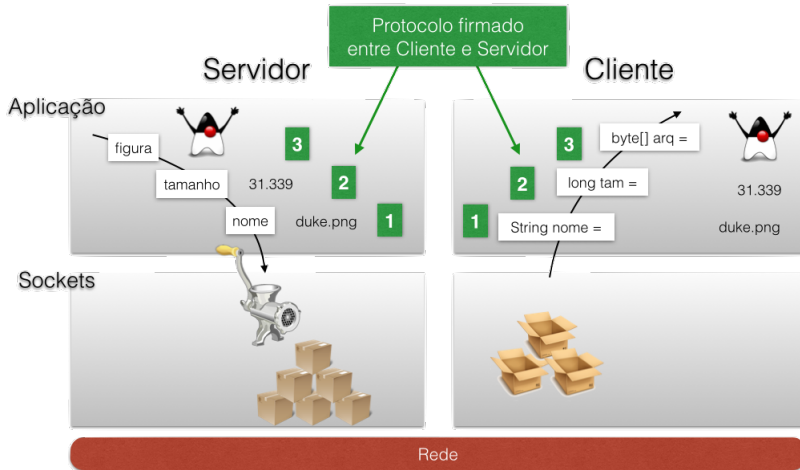
Cliente e Servidor com Sockets TCP



Cliente e Servidor com Sockets UDP



Protocolo definido entre Cliente e Servidor



Protocolo definido entre Cliente e Servidor – em Java

Cliente conecta e o servidor envia a primeira mensagem

Servidor

```
OutputStream os = conexao.getOutputStream();  
DataOutputStream dout = new DataOutputStream(os);  
  
dout.writeUTF("duke.png"); // envio da mensagem 1  
dout.writeLong(31.339);    // envio da mensagem 2  
dout.write(blocos);        // envio da mensagem 3
```

Cliente

```
Socket conexao = new Socket("127.0.0.1", 1234);  
InputStream is = conexao.getInputStream();  
DataInputStream dis = new DataInputStream(is);  
  
String nome = dis.readUTF(); // mensagem 1  
long tamanhoArq = dis.readLong(); // mensagem 2  
byte[] arq = dis.read(blocos, 0, tamanhoArq); // mensagem 3
```

Transmissão de dados pela rede

Transmissão de dados pela rede

- Transmissão de dados pela rede requer um acordo prévio entre cliente e servidor para que ambos possam **representar os dados corretamente** em seus ambientes
 - Mensagens são transmitidas como **fluxos de bytes**
- Os **processos cliente** e **servidor** podem ser executados em **diferentes arquiteturas** de máquinas (i.e. Intel *versus* PowerPC)
- Máquinas distintas podem ter diferença
 - Na ordenação de bytes
 - Na quantidade de bytes para representar inteiros
 - Na representação de valores reais
 - Na codificação de caracteres (i.e. ASCII *vs* UTF-8 *vs* Windows-1252)

Heterogeneidade de representação de dados

Extremidade (*endianness*)

Ordem usada para representar valores numéricos na memória ou quando transmitidos pela rede

- **big-endian** – bytes armazenados em ordem decrescente do seu peso numérico
 - byte mais significativo é armazenado primeiro
- **little-endian** – bytes armazenados em ordem crescente do seu peso numérico
 - byte menos significativo é armazenado primeiro

Heterogeneidade de representação de dados



- Arquitetura x86-64 usa little-endian (convenção da Intel)
- Poucas arquiteturas (PowerPC antigo, Xilinx MicroBlaze, etc) usam o big-endian, porém foi convencionado pela IETF para ser usado pelos protocolos da Internet
 - Cabeçalho IP usa big-endian
- **bi-endian** podem operar com ambas (ARM, MIPS, IA-64)

Quantidade de bytes para representar um número inteiro

- Em Java o tipo primitivo `long` sempre ocupa 8 bytes
- Na linguagem C depende da arquitetura de máquina

```
#include<limits.h>
int main(void){
    int i; long l;
    printf("%ld, %d",sizeof(i),INT_MAX);
    printf("%ld, %ld",sizeof(l),LONG_MAX);
}

/* Resultado em uma máquina de 64bits */
4, 2147483647
8, 9223372036854775807

/* Resultado em uma máquina de 32bits */
4, 2147483647
4, 2147483647
```

Codificação de caracteres

- Tipo primitivo `char` em C ocupa 1 byte para representar caracteres ASCII
- UTF-8 é de codificação binária de tamanho variável (1 a 4 bytes) que permite representar qualquer caractere universal
 - 1 byte – para representar os 128 caracteres ASCII
 - 2 bytes – para representar caracteres latinos com diacríticos (i.e. acentos)
 - 3 bytes – para alfabetos Grego, Cirílico, Armênio, Hebraico, Sírio e Thaana
 - 4 bytes – para representar outros caracteres

Nota

UTF-8 é o formato de codificação padrão da maioria dos sistemas operacionais e também de todos os protocolos da Internet padronizados pela IETF (RFC 3629)

Prática: ferramenta netcat

Ferramenta netcat

- Servidor aceita uma única conexão e depois encerra

```
# Ouvir na porta 1234/TCP  
nc -l -p 1234
```

```
# Conectar no servidor  
nc IP-do-servidor 1234
```

- Cliente enviando um arquivo para o servidor

```
nc -l -p 1234 > arquivo.txt
```

```
nc IP-do-servidor 1234 < arq.txt
```

- Usando UDP

```
nc -u -l -p 1234
```

```
nc -u IP-do-servidor 1234
```

- Um simples servidor web

```
echo -e 'HTTP/1.1 200 OK\n\n <html><h1>Ola mundo</h1></html>' | nc -l -p 1234
```

Curiosidade

- UTF-8 foi criado por Ken Thompson, juntamente com Rob Pike, em um jogo americano (porta prato) durante o jantar em um restaurante em setembro de 1992 em New Jersey¹
- Ken Thompson e Dennis Ritchie foram responsáveis por criar a linguagem de programação B, predecessora da linguagem C
 - Ambos também atuaram no desenvolvimento do sistema operacional Unix
- Dennis Ritchie é um dos autores da linguagem C
- Ken Thompson também atuou no desenvolvimento da linguagem Go na Google

¹ <https://www.cl.cam.ac.uk/~mgk25/ucs/utf-8-history.txt>

Leitura obrigatória

- Leitura do capítulo 4 do livro *Distributed Systems* 3rd edition

Aulas baseadas em



COULORIS, George; DOLLIMORE, Jean; KINDBERG, Tim. **Sistemas Distribuídos: Conceitos e projeto**. 5. ed.: Bookman, 2013.

<https://app.minhabiblioteca.com.br/books/9788582600542/>.



KRZYZANOWSKI, Paul. **Distributed Systems – Rutgers University**. 2020.



TANENBAUM, Andrew S; STEEN, Maarten van. **Sistemas Distribuidos: Princípios e paradigmas**. 2. ed.: Prentice Hall, 2008.